BPF-GNN: A Multi-Granularity Feature Extraction Model Using Graph Neural Networks for Encrypted Traffic Classification

Guolong Li, Yuan Gao, Jiongjiong Ren, and Shaozhen Chen

***Abstract*—Encrypted traffic classification is crucial for critical network management tasks such as traffic type identification, resource allocation, and risk mitigation, especially given that encrypted traffic has become the dominant form of modern network communication. However, existing classification methods are typically confined to single-level feature extraction, failing to capture the multi-granularity information inherent in traffic and thus limiting their ability to characterize complex encrypted traffic patterns. To address this issue, this paper proposes BPF-GNN, a hierarchical graph feature extraction model for encrypted traffic classification. The model enables multi-granularity feature learning by constructing a three-tier graph structure (Byte-, Packet-, and Flow-level). It sequentially extracts discriminative information inherent in each granularity level and accumulates multi-dimensional traffic characteristics, significantly improving the classification accuracy of encrypted traffic. Experiments on the ISCX-VPN2016, ISCX-Tor2016, USTC-TFC2016, and MIRAGE-2024 datasets demonstrate that BPF-GNN outperforms existing methods, validating the effectiveness and superiority of the proposed hierarchical multi-granularity feature extraction approach.**

***Index Terms*—Encrypted traffic classification, deep learning, graph neural networks, multi-granularity feature extraction.**

I. Introduction

NETWORK traffic classification is a fundamental and critical task in network research, which is essential for Quality of Service (QoS), malware detection, and intrusion detection [1]. In recent years, with the widespread use of encryption technology, the privacy and anonymity of network traffic have been significantly enhanced. However, the prevalence of encryption technology has also brought new challenges to network traffic analysis. Traditional traffic classification methods, such as port-based techniques and Deep Packet Inspection (DPI) [2], [3], [4], have seen a marked decrease in effectiveness when dealing with encrypted traffic. Port-based methods have become ineffective due to the extensive use of dynamic ports, and DPI not only has high computational overhead when processing encrypted traffic but also struggles to extract valid information.

Received 12 June 2025; revised 8 January 2026 and 2 March 2026; accepted 3 March 2026. Date of publication 6 March 2026; date of current version 16 March 2026. This work was supported by the National Natural Science Foundation of China under Grants No. 62206312. The associate editor coordinating the review of this article and approving it for publication was F. Musumeci. *(Corresponding author: Yuan Gao.)*

The authors are with the Information Engineering University, Zhengzhou 450001, China (e-mail: liguolongx@163.com; yuanmiao202311@163.com; jiongjiong_fun@163.com; chenshaozhen@vip.sina.com).

Digital Object Identifier 10.1109/TNSM.2026.3671203

To address these challenges, researchers have proposed a variety of methods based on machine learning (ML) and deep learning (DL) [5], [6], [7], [8], [9]. Early machine learning methods relied on manually designed statistical features, which achieved some success to a certain extent. However, these methods required manual feature engineering, and their accuracy largely depended on the quality of the feature engineering. In recent years, the rise of deep learning technology has provided new solutions for encrypted traffic classification. Deep learning models can automatically extract features from raw traffic data, avoiding cumbersome feature engineering, and demonstrating strong learning capabilities and generalization performance. However, a critical limitation persists: most deep learning frameworks based on traditional neural networks such as Convolutional Neural Network (CNN) and Recurrent Neural Network (RNN) process traffic data in Euclidean space, neglecting the inherent non-Euclidean nature of network traffic. This oversight may lead to the loss of valuable information [10].

Graph Neural Networks (GNNs), designed for non-Euclidean data, provide a natural solution for modeling complex traffic relationships. As a result, researchers are increasingly interested in GNNs for encrypted traffic classification. However, most current GNN-based methods [10], [11], [12], [13], [14] focus only on feature extraction at a single level of the graph (network traffic can be divided into three levels from low to high: byte, packet, and flow), neglecting the fact that there is unique potential available information at each granularity level. Additionally, in deep learning methods that use raw bytes as input, headers and payloads are often treated as a single learning entity without distinguishing their different representations and functions, which may lead to some degree of semantic confusion [15].

In response to the aforementioned issues, this paper proposes BPF-GNN, a hierarchical graph feature extraction model that constructs byte-, packet-, and flow-level graphs for multi-granularity feature extraction. The model consists of three main parts, as shown in Fig. 1: byte-level graph construction and packet-level representation learning, packet-level graph construction and flow-level representation learning, flow-level graph construction and classification. First, at the byte level, we employ a dual embedding layer to separately construct byte correlation graphs for packet headers and payloads, mining the potential correlations between bytes in packet headers and payloads. A feature extractor composed of multi-layer

1932-4537 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:17 UTC from IEEE Xplore. Restrictions apply.

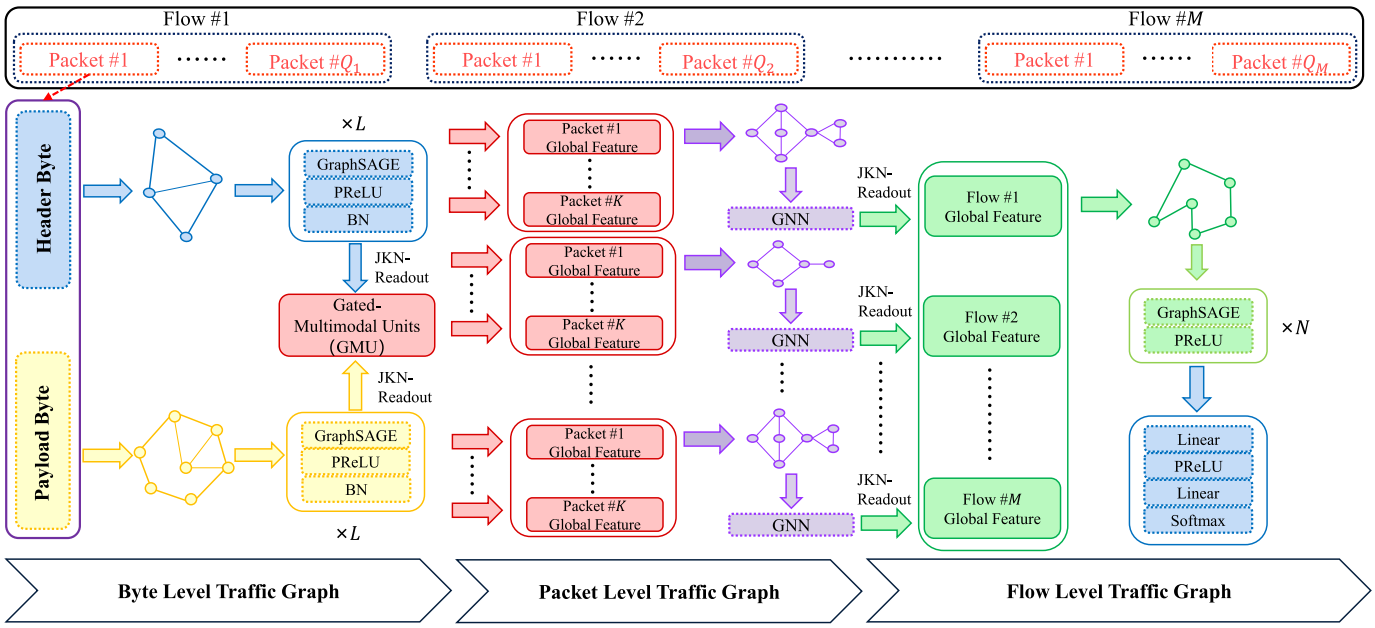


Fig. 1. BPF-GNN model architecture.

GNNs performs graph representation learning to obtain the representation vectors of headers and payloads, respectively. These two types of graph representation vectors are then fused via a Gated Multimodal Unit (GMU) to generate a joint representation. Second, at the packet level, we construct a Traffic Interaction Graph (TIG) [13] using the joint representations as node features to extract interaction information between packets. Similarly, we use multi-layer GNNs to conduct graph representation learning and derive the overall representation vector for each flow. Finally, at the flow level, we select the Top-k most similar flows for each flow and establish connections between flow nodes, thereby supplementing similarity information among flows and enriching the flow-level feature representations for classification.

Experiments on the ISCX-VPN2016, ISCX-Tor2016, USTC-TFC2016, and MIRAGE-2024 datasets demonstrate that BPF-GNN effectively distinguishes fine-grained categories of encrypted traffic, achieving significantly higher classification accuracy than baselines. The contributions of our work are threefold:

1) We propose a method BPF-GNN for hierarchically extracting features at three different levels, rather than only focusing on feature extraction at a single level. This allows us to extract the potential information contained at the byte, packet, and flow levels, thereby enriching the representation of the flow-level features used for classification.
2) We start feature extraction from the basic level (byte-level) of traffic to learn deeper feature representations and handle the header and payload bytes of each packet separately to avoid semantic confusion. Furthermore, after the GNN layer, we use GMU to obtain the joint representation of the packet header and payload. This approach enables the model to leverage the complementary information present in the header and payload.
3) To evaluate the performance of BPF-GNN, we compare it with several existing methods on four public datasets. The results demonstrate that BPF-GNN significantly outperforms these existing methods. This remarkable performance can be attributed to the effectiveness of the multi-granularity feature extraction approach.

II. Considered Scenario

A. Application Scenario

Encrypted traffic classification serves as a pivotal technical support for modern network operation and security governance, with its application scenarios covering enterprise intranets, campus networks, cloud computing platforms, and public network access gateways. These scenarios are characterized by diverse network architectures (including wired/wireless integrated networks, LANs, and WANs) and heterogeneous user groups (such as enterprise employees engaged in remote office, campus users with mixed service demands, and individual consumers accessing public services), resulting in complex and variable encrypted traffic patterns.

The core application objectives in these scenarios include the following two dimensions: first, refined network resource management, which requires distinguishing fine-grained application types (e.g., chat, email, file transfer, streaming media) in encrypted traffic to optimize bandwidth allocation and QoS guarantee strategies; second, targeted security risk prevention, which involves identifying abnormal encrypted traffic (such as malicious traffic disguised by encryption) to block hidden attack channels and comply with network security management regulations.

In practical deployment, the classification system is required to operate in a passive and non-intrusive mode. It only collects and analyzes traffic data, without decrypting payloads or mod-

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:17 UTC from IEEE Xplore. Restrictions apply.

ifying packet structures. This design ensures compliance with data privacy protection specifications and avoids interference with normal network communication.

B. Task Description

Encrypted traffic classification is defined as a supervised learning task that maps encrypted network flow samples to predefined category labels based on their inherent features. This task focuses on two levels of classification objectives aligned with practical application demands:

1) Fine-grained application type classification: Distinguishing specific service types within encrypted traffic, such as chat, email, file transfer, peer-to-peer (P2P), streaming media (audio/video), and voice over IP (VoIP). This level of classification provides a basis for refined network resource optimization and legitimate service management, enabling targeted allocation of network bandwidth and QoS guarantees.
2) Normal/abnormal traffic discrimination: Identifying whether encrypted traffic belongs to legitimate user behavior or malicious activities (e.g., botnet communication, malware propagation, unauthorized data exfiltration). This objective is crucial for network security defense, helping to detect hidden threats and maintain the security and compliance of network operations.

Formally, let $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N\}$ denote a dataset consisting of N encrypted traffic samples, where each sample $\mathbf{x}_i$ corresponds to a bidirectional network flow defined by a 5-tuple (source IP, destination IP, source port, destination port, transport layer protocol). Each flow sample is composed of a sequence of packets, including header metadata and encrypted payloads. Let $\mathbf{Y} = \{\mathbf{y}_1, \mathbf{y}_2, \ldots, \mathbf{y}_N\}$ represent the corresponding label set, where $\mathbf{y}_i \in \{1, 2, \ldots, C\}$ (C is the number of target categories).

The core challenge of this task lies in overcoming the semantic obscurity caused by encryption. The model must mine discriminative information from multi-granularity traffic representations. The ultimate goal is to learn a mapping function $f : \mathbf{X} \rightarrow \mathbf{Y}$ that achieves high accuracy, precision, recall, and F1-score on diverse encrypted traffic scenarios.

III. Related Work

A. Machine Learning-Based Methods

In recent years, machine learning-based methods have achieved promising results in the field of traffic classification. These methods combine discriminative features extracted from encrypted traffic with machine learning algorithms. They first extract feature information from the traffic and then input these features into machine learning models for classification.

A large number of studies focus on mining statistical features inherent in traffic to build classification models. Finamore et al. [16] proposed KISS, which extracts payload statistical features via the Chi-Square test and combines geometric distance or Support Vector Machines to address the classification needs brought by the growth of UDP traffic. Anderson et al. [6] compared six machine learning models, and confirmed the critical role of feature engineering and enhanced features in improving classification performance. Zaki et al. [17] proposed GRAIN, which realizes three-level granular multi-label classification of encrypted traffic based on statistical features of packet payload lengths and cascaded Random Forest classifiers. Mohanty et al. [18] proposed a stacking ensemble model and an autoencoder-based defense mechanism to solve the problems of high False Positive Rate and vulnerability to adversarial attacks in darknet traffic classification.

Another line of research directly takes raw bit/byte streams as input to machine learning models. Yun et al. [19] proposed Securitas, a protocol identification system that leverages the skewed frequency-rank distribution of protocol trace n-grams, extracts statistical message formats by clustering semantically identical n-grams, and identifies protocols from raw network traces without prior protocol knowledge. Wang et al. [20] proposed ProHacker, a nonparametric approach that extracts protocol keywords via a Bayesian statistical model and classifies protocol traces using semi-supervised learning. Xiao et al. [21] proposed the Dynamic Multiple Traffic Classification System (DMTCS), which incorporates time-based protocol information distribution and applies topic models to cluster packets, enabling dynamic network stream classification without prior application knowledge. Tang et al. [22] proposed MGrC based on granular computing, which defines traffic flow granules, explores inter-granule correlations, and establishes structural granules to capture inherent packet relationships.

However, the main drawback of these methods lies in the generally shallow structure of machine learning models, which makes it difficult to capture the complex correlation features in encrypted traffic and results in insufficient adaptability to high-dimensional traffic data with variable patterns [23].

B. Deep Learning-Based Methods

Thanks to the rapid development of deep learning technology, researchers have begun to apply deep learning techniques to the field of network traffic classification. The earliest methods to emerge were based on traditional neural networks. Wang et al. [8] transformed traffic into images and proposed a CNN-based method for encrypted traffic classification. Bu et al. [24] proposed a model with deep and parallel network-in-network (NIN) structures, which enhances local modeling via micro-networks after convolution layers and reduces parameters with global average pooling. Liu et al. [25] proposed FS-Net based on RNN, which leverages a multi-layer encoder-decoder structure to mine flow sequential characteristics and a reconstruction mechanism to enhance feature effectiveness. Ren et al. [26] proposed Tree-RNN, which divides large classification tasks into small ones via tree structure and deploys specific classifiers for each sub-task, realizing end-to-end feature learning. These methods avoid tedious feature engineering in traditional machine learning, yet rely on traditional neural networks such as CNN and RNN to process Euclidean space data. They only efficiently handle regular 1D or 2D data, ignore the intrinsic non-Euclidean traits of network traffic, and lead to the loss of valuable information [10].

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:17 UTC from IEEE Xplore. Restrictions apply.

In recent years, GNNs have become a hot topic in the field of deep learning. These networks can integrate deep learning with graph structures, making up for the shortcomings of the aforementioned traditional neural network-based classification methods.

Researchers have attempted to combine deep learning-based methods with graph-based methods to improve the effectiveness of traffic classification. Zheng et al. [11] proposed a graph construction method that combines traffic trace graph with statistical features, which can achieve certain performance improvements under low labeling rates, but its accuracy is relatively low. Sun et al. [12] constructed a graph structure based on the K-Nearest Neighbors (KNN) method and used a fusion neural network of graph convolution and autoencoder to extract graph features. Shen et al. [13], leveraging the slightly different characteristics of traffic in terms of bursts, proposed TIG, which achieved good classification results on Decentralized Applications (DApps). Huoh et al. [10] construct a graph structure using the raw bytes of packets, the sequential relationships between packets, and packet metadata. The raw byte information is utilized as node features, and edges were generated according to the chronological order of each packet's timestamp. Additionally, five common meta-features are included as global features of the graph. They design a GNN to learn representations for nodes, edges, and global features. Diao et al. [14] proposed EC-GCN based on multi-scale graph convolutional neural networks, which encodes traffic flows into graph representations via a lightweight layer relying only on metadata and extracts multi-graph representations through a graph pooling and structure learning layer. Yang et al. [27] proposed MTSecurity, which integrates Transformer and GNN technologies, constructs the Malicious Traffic Interaction Graph (MTIG) based on client-server interactions, and fuses raw byte features with graph-based interaction features. Shi et al. [28] proposed BPF-DAG, which constructed a global IP-level communication graph and adopted Transformer and a customized DiESAGE to extract packet-level temporal features and flow-level topological features, respectively.

A common issue with the aforementioned methods is that they focus only on feature extraction at one level of the graph, neglecting the fact that there is unique potential available information at each granularity level. For example, the methods of Zheng et al. and Sun et al. both focus only on feature extraction at the flow level, while the methods of Shen et al. and Huoh et al. focus only on feature extraction at the packet level. Furthermore, the model proposed by Shi et al. only focuses on the temporal relations at the packet level and the topological relations at the flow level, without considering the relational information among bytes within a packet, nor does it achieve real feature modeling at the byte level.

Therefore, based on the above observations, we propose a hierarchical feature extraction model BPF-GNN that constructs byte-, packet-, and flow-level graph structures for multi-granularity feature extraction.

IV. Methodology

In this section, we provide a detailed description of our method as well as the overall model architecture. The overall process of our method is shown in Fig. 2, which is divided into five stages: data preprocessing, byte-level graph construction and packet-level representation learning, packet-level graph construction and flow-level representation learning, flow-level representation enrichment and classification.

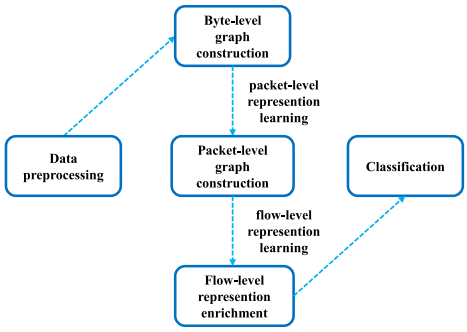


Fig. 2. The overall process of our method.

A. Data Preprocessing

Assuming that the raw traffic data we obtain is in the PCAP format, due to the varying lengths of the raw traffic data and the presence of interfering information, we need to preprocess the data to transform it into uniformly formatted input data. The data preprocessing consists of five steps: traffic splitting, flow filtering, anonymization, and extraction of raw traffic bytes from packet header/payload.

1) Traffic Splitting: First, we need to split the continuous traffic into multiple discrete units. The most commonly used method in current research is to divide the raw traffic data into bidirectional flows. A flow refers to all packets that share the same five-tuple information (source IP, source port, destination IP, destination port, and transport layer protocol). Following this step, the raw traffic data is segmented into a set of flows $F = \{f_1, \ldots, f_N\}$, where N represents the size of F.

2) Flow Filtering: We filter out flows in which all of packets have no payload. This is because these flows do not contain any actual transmission content or data, and thus cannot be used to construct the corresponding graph structure. Furthermore, we also filter out flows with an excessively large number of packets. Specifically, flows with more than 10,000 packets are defined as "excessively large flows" and filtered out. This is because such flows are mostly caused by poor network conditions or other underlying factors, which introduce massive noise via bad and retransmitted packets that may interfere with subsequent graph construction and feature learning. Therefore, such flows are typically regarded as abnormal samples and are removed. After this step, the flow set F is filtered into set $F' = \{f'_1, \ldots, f'_M\}$, where M represents the size of F' and $M \leq N$.

3) Anonymization: The five-tuple information of each packet in each flow f'_i is directly removed, as this information may interfere with the classification results of encrypted traffic.

4) Extraction of Raw Traffic Bytes: For each flow f'_i, select the first K packets and extract the first m bytes of the packet header and the first n bytes of the payload as the raw byte

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:17 UTC from IEEE Xplore. Restrictions apply.

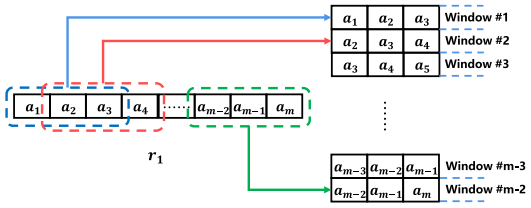


Fig. 3. An example of sliding window with window size of 3.

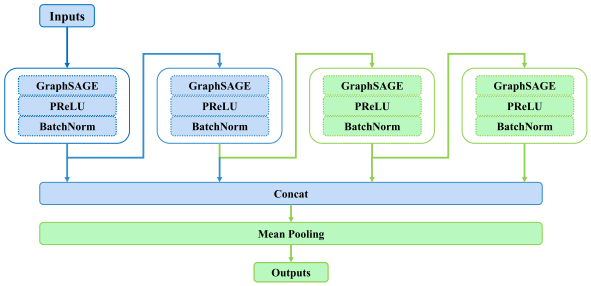


Fig. 4. Structure of GNN for packet-level representation learning.

sequences for header/payload. Byte streams longer than m/n bytes are truncated, while those shorter than m/n bytes are padded with $0xFF$ at the end to reach the required length. If the number of packets in flow f_i' is less than K, the flow is padded with several m/n-length byte sequences filled with $0xFF$.

Consequently, the packet header/payload information of each flow f_i' is transformed into a byte matrix $H_i \in \mathbb{R}^{K\times m}/P_i \in \mathbb{R}^{K\times n}$ to represent it.

B. Byte-Level Graph Construction and Packet-Level Representation Learning

In this section, we introduce the process of constructing byte-level graph and packet-level representation learning. To avoid confusion in meaning, we perform representation learning for packet header and payload independently [15], [29], [30]. Next, we present a detailed description.

1) Byte-Level Graph Construction: In this section, our goal is to transform each row of the packet header/payload matrices obtained in Section IV-A into a byte-level graph $G_{byte} = \{V_{byte}, E_{byte}\}$ to serve as input data for the graph neural network, where V_{byte} and E_{byte} are the sets of vertices and edges. The graph construction method refers to the approach proposed in Reference [15]. Below, we use the first row r_1 of the packet header byte matrix H_1 for flow f_1' as an example to illustrate the process of constructing the corresponding byte-level graph.

Here, we assume that

$$r_1 = \{a_1, \ldots, a_m\}, a_i \in [0, 255], i = 1, \ldots, m.$$

Each element in V_{byte} represents a byte of r_1. Note that all bytes with the same value share the same node, so the number of nodes in G_{byte} does not exceed 256, which ensures that the size of the byte-level graph remains relatively small.

To construct the edge set E_{byte}, we first process the input byte sequence r_1 using a sliding window (as shown in Fig. 3). This step is designed to capture local contextual correlations between bytes, which is a key property for distinguishing encrypted traffic patterns. Semantic associations in packet headers and payloads are typically concentrated in adjacent or short-range byte sequences, making these local correlations particularly valuable for classification. Specifically, we set a window of fixed size (e.g., 3 bytes in Fig. 3) and slide it across r_1 with a step size of 1: each window defines a local context, and bytes within the same window are treated as co-occurring elements. This approach avoids the over-dense graph structure that would result from connecting all bytes chronologically, while focusing on meaningful short-range byte relationships.

Next, we count the frequency of each byte appearing in various windows and record the number of times byte pairs co-occur within the same window. This statistical information will be used to calculate the strength of association between bytes. To quantify this strength of association, we use Pointwise Mutual Information (PMI) [31] as a measure. The formula for calculating PMI is as follows:

$$PMI(x; y) = \log \frac{p(x, y)}{p(x)p(y)}.$$

PMI measures the probability of two bytes co-occurring in a window relative to the probability of their independent occurrences. If the PMI value is positive, it indicates that the two bytes have a strong association in context, and thus an edge connection is established between the nodes corresponding to these two bytes; if the PMI value is negative or zero, it is considered that there is no significant association between them.

2) Packet-Level Representation Learning: Following the methodology established in Section IV-B.1, we generate byte-level graphs for both packet headers and payloads. To perform feature extraction on these graphs, we implement a four-layer GraphSAGE architecture [15], as detailed in Fig. 4. GraphSAGE [32] is an GNN framework that learns node embeddings by sampling and aggregating features of local neighbor nodes. This framework possesses a strong capability to capture local structural correlations and can effectively mine the local relevance between bytes in byte-level graphs. The core process of GraphSAGE consists of two sequential steps: neighbor feature aggregation and node embedding update. For each node v, the neighbor aggregation operation is defined as:

$$\text{AGGREGATE}\left(\{q_u \mid u \in \mathcal{N}(v)\}\right),$$

where we adopt mean aggregation to achieve stable and reliable feature learning. The updated node embedding is then calculated by the following formula:

$$q_v' = \sigma\left(W^{(l)} \cdot \text{CONCAT}\left(q_v, \text{AGGREGATE}(\cdot)\right)\right),$$

which concatenates the node's own features with the aggregated neighbor information, thereby preserving both local details and contextual associations simultaneously.

Formally, for each node $v \in V_{byte}$ in the graph G_{byte}, the feature aggregation process can be described as follows. Let $q_v^{(l-1)}$ denote the feature vector of node v at layer $l-1$, and

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:17 UTC from IEEE Xplore. Restrictions apply.

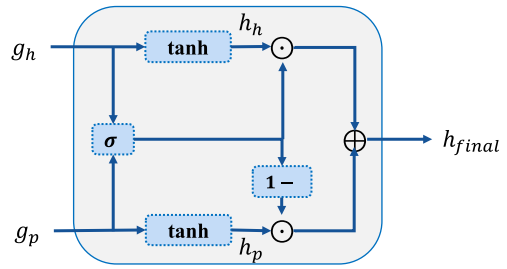


Fig. 5. The simplified version of GMU.

$\mathcal{N}(v)$ represent the set of neighboring nodes of v. The message from the neighbors is computed as:

$$p^{(l)} = \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} q_u^{(l-1)},$$

where $|\mathcal{N}(v)|$ is the cardinality of the neighbor set. The updated feature vector for node v at layer l is then obtained by concatenating its current feature vector with the aggregated neighbor message, followed by a non-linear transformation:

$$q_v^{(l)} = \sigma\left(W^{(l)} \cdot \text{CONCAT}\left(q_v^{(l-1)}, p^{(l)}\right)\right).$$

Here, $W^{(l)} \in R^{d_{l-1} \times d_l}$ is the learnable weight matrix at layer l, CONCAT$(\cdot)$ denotes the concatenation operation, and $\sigma(\cdot)$ is the activation function. We utilize the Parametric ReLU (PReLU) [33] as the activation function, defined as:

$$\sigma(x) = \begin{cases} x & \text{if } x > 0, \\ ax & \text{if } x \leq 0, \end{cases}$$

where a is a learnable parameter that introduces channel-wise attention by scaling negative values differently. To enhance the stability and performance of the model, we apply Batch Normalization (BN) [34] to the updated feature vectors:

$$q_v^{(l)} = \text{BN}\left(q_v^{(l)}\right).$$

To address the over-smoothing issue and capture multi-scale structural information, we adopt a strategy inspired by Jumping Knowledge Networks (JKN) [35]. Specifically, we concatenate the feature vectors from the first four layers to form the final feature representation for each node v:

$$q_v = \text{CONCAT}\left(q_v^{(1)}, q_v^{(2)}, q_v^{(3)}, q_v^{(4)}\right).$$

Subsequently, we compute the global feature vector g_{packet} by applying mean pooling over all node feature vectors:

$$g_{packet} = \frac{1}{|V_{byte}|} \sum_{v \in V_{byte}} q_v,$$

where $|V_{byte}|$ denotes the number of nodes in the graph. This global feature vector g_{packet} encapsulates the structural characteristics of the entire graph.

So far, we have been able to obtain packet-level representation vectors for both the header and payload of each packet.

3) Representation Fusion: Having derived packet-level representation vectors for both headers and payloads, we integrate these representations into a unified vector through a streamlined GMU [36]. This mapping operation projects the header and payload features into a joint latent space, enabling subsequent packet-level graph construction. The GMU architecture is detailed in Fig. 5.

Let g_h and g_p represent the packet-level representation vectors extracted from the traffic header graphs and traffic payload graphs, respectively. The GMU computes a gating vector to control the information between header and payload representation vectors.

First, we transform the header and payload representation vectors using hyperbolic tangent (tanh) activation functions:

$$h_h = \tanh\left(W_h \cdot g_h\right),$$
$$h_p = \tanh\left(W_p \cdot g_p\right),$$

where W_h and W_p are the learnable weight matrices for the header and payload modalities, respectively. The tanh function ensures that the transformed representation vectors are within the range $[-1, 1]$.

Next, we compute the gating vector z using a sigmoid activation function:

$$z = \sigma\left(W_z \cdot \text{CONCAT}(g_h, g_p)\right),$$

where W_z is the learnable weight matrix for the gating mechanism, and $\sigma(\cdot)$ denotes the sigmoid activation function. The gating vector z determines the contribution of each modality to the final representation.

The final fused representation h_{final} is obtained by combining the gated vectors from header and payload representation vectors:

$$h_{\text{final}} = z \odot h_h \oplus (1 - z) \odot h_p,$$

where $\odot$ denotes the element-wise multiplication operation, and $\oplus$ represents element-wise addition. This formulation allows the GMU to adaptively weigh the contributions of the header and payload feature vectors.

By incorporating this simplified GMU, we provide an efficient and effective framework for fusing header and payload representation vectors. This approach enables the model to leverage the complementary information present in header and payload.

C. Packet-Level Graph Construction and Flow-Level Representation Learning

Having acquired the overall representation vector for each packet, which is the packet-level representation vector, we proceed to construct graph structures at the packet level to uncover potential useful information that is implicit at the packet level, such as interaction feature information between packets. It is also important to note that our primary focus in this paper is on the flow-level classification task. Therefore, we also need to extract the global features of each packet-level graph, which is the learning of flow-level representations.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:17 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Packet-Level Graph Edge Set Construction

Input: Vertex set V.
Output: The packet-level graph edge set E.
Initialize: $E = \phi$.
1: Split V into bursts $B = (b_1, \ldots, b_S)$ based on packet direction
2: **for** each burst $b_i \in B$ **do**
3: **if** b_i has more than one packet **then**
4: **for** each pair of consecutive packets (v_j, v_{j+1}) in b_i **do**
5: Add bidirectional edges between v_j and v_{j+1} to E
6: **end for**
7: **end if**
8: **end for**
9: **for** each pair of consecutive bursts (b_i, b_{i+1}) **do**
10: **if** both b_i and b_{i+1} have only one packet **then**
11: Add bidirectional edges between the packets in b_i and b_{i+1} to E
12: **else**
13: Add bidirectional edges between the first packets of b_i and b_{i+1} to E
14: Add bidirectional edges between the last packets of b_i and b_{i+1} to E
15: **end if**
16: **end for**
17: **return** E

1) Packet-Level Graph Construction: Generally speaking, the interaction patterns between the two parties in network traffic communication are unique and remain unchanged regardless of whether the traffic itself is encrypted and protected by protocols. Therefore, at the packet level, we opt to construct the TIG [13] to learn this unique information.

Our goal is to construct the corresponding packet-level graph $G_{packet} = \{V_{packet}, E_{packet}\}$ for each flow, where the number of nodes in each graph is K, and the node features of each node are the vectors h_{final} extracted as described in Section IV-B. It should be specially noted that the node features of the Traffic Interaction Graph (TIG) in [13] are packet lengths. To adapt to our model, we replace the node features in the original TIG method with the extracted packet-level representation vectors.

The method for constructing the edge set E_{packet} in the traffic interaction graph is shown in Algorithm 1, where a burst is defined as a sequence of consecutive packets transmitted in the same direction within a short time interval [37]. Lines 2-8 of the algorithm are for adding intra-burst edges, which are the connections between vertices within the same burst. Lines 9-16 are for adding inter-burst edges, which establish connections between two adjacent bursts. More specifically, the first and last vertices in each burst are connected to the corresponding first and last vertices in the next burst.

2) Flow-Level Representation Learning: After constructing the packet-level traffic graph, we need to perform representation learning for the graph features. Here, we still use the GraphSAGE layer as mentioned in Section IV-B.2, only this time we stack two layers. Since this paper focuses on flow-level classification tasks, we still need to extract global features to obtain flow-level representations. Specifically, we concatenate the vectors from the first two layers to form the final representation vectors for each node v:

$$l_v = \text{CONCAT}\left(l_v^{(1)}, l_v^{(2)}\right).$$

Subsequently, we compute the global feature vector g_{flow} by applying mean pooling over all node feature vectors:

$$g_{flow} = \frac{1}{K} \sum_{v \in V_{packet}} l_v.$$

The learned flow-level representations g_{flow} then contain two types of information:

1) **Packet-level representations learned from byte-level graphs.** Since the node features in the packet-level graph use global packet features extracted from the byte-level, the extracted flow-level features include potential relationship information between bytes;
2) **Interaction information unique to the packet level.** As mentioned before, the interaction pattern information of packets is unique and is not affected by whether the traffic itself is encrypted or not. After representation learning, the extracted flow-level representations currently include this important information to better perform subsequent classification tasks.

D. Flow-Level Representation Enrichment and Classification

After completing the two-step representation learning process described in Sections IV-B and IV-C, we have obtained flow-level representations. However, we believe that additional flow-level similarity information can be incorporated to further enhance the representation of flows. To achieve this, we propose constructing the flow-level graph where the flow-level representations obtained in Section IV-C are used as node features.

Let $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_M\}$ represent the set of flow-level representations extracted as described in Section IV-C, where $\mathbf{x}_i \in \mathbb{R}^d$ is the representation vector of the i-th flow, and M is the total number of flows. These representation vectors are derived from the previous steps and capture the essential characteristics of each flow. It should be specifically noted that, in order to keep the size of the flow-level graph relatively small, we construct a graph every BatchSize (BS) flows, instead of constructing a large graph using all M flows. This choice avoids excessive computational and memory overhead caused by a single large-scale graph when M is large, ensuring the model's scalability to large datasets while maintaining the local similarity structure within each batch.

To establish connections between flows, we use the Euclidean distance as the similarity metric. The Euclidean distance between two flow representation vectors $\mathbf{x}_i$ and $\mathbf{x}_j$ is defined as:

$$d(\mathbf{x}_i, \mathbf{x}_j) = \|\mathbf{x}_i - \mathbf{x}_j\|_2 = \sqrt{\sum_{k=1}^{d} (x_{ik} - x_{jk})^2},$$

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:17 UTC from IEEE Xplore. Restrictions apply.

where x_{ik} and x_{jk} are the k-th components of the representation vectors $\mathbf{x}_i$ and $\mathbf{x}_j$, respectively.

For each flow $\mathbf{x}_i$, we select the Top-k most similar flows based on the Euclidean distance. Specifically, we compute the distances between $\mathbf{x}_i$ and all other flows $\mathbf{x}_j$ (where $j \neq i$), and then select the k flows with the smallest distances. These k flows are considered the most similar neighbors of $\mathbf{x}_i$.

Using the Top-k similarity information, we construct a flow-level graph $G_{flow} = (V_{flow}, E_{flow})$, where:

- $V_{flow} = \{v_1, v_2, \ldots, v_{BS}\}$ is the set of nodes, each representing a flow.
- E_{flow} is the set of edges, where an edge (v_i, v_j) exists if $\mathbf{x}_j$ is one of the Top-k most similar neighbors of $\mathbf{x}_i$.

To enrich the flow-level representations, we apply two layers of GraphSAGE on the constructed graph. By applying two layers of GraphSAGE, each node can aggregate information from its Top-k most similar neighbors, thereby enhancing the flow-level representations.

This graph construction approach creates more connections between flows of the same application type, which allows each node to aggregate information from more neighbors of similar types. This enhances the flow-level representations and improves the accuracy of the classification task.

Finally, we use a fully connected layer to perform the ultimate classification task. The output of the second GraphSAGE layer is passed through the fully connected layer, which is defined as:

$$\mathbf{y}_i = \text{softmax}\left(\mathbf{W}^c \cdot \mathbf{h}_i^{(2)} + \mathbf{b}^c\right),$$

where:

- $\mathbf{h}_i^{(2)}$ is the feature representation of node v_i after the second GraphSAGE layer.
- $\mathbf{W}^c$ and $\mathbf{b}^c$ are the learnable weight matrix and bias vector of the fully connected layer, respectively.
- $\mathbf{y}_i$ is the predicted probability distribution over the classes for node v_i.

To optimize the model parameters, we utilize the cross-entropy loss function to measure the discrepancy between the predicted probabilities and ground-truth labels. The loss is computed as:

$$\mathcal{L} = -\sum_{c=1}^{C} \mathbf{y}_{i,c}^{\text{true}} \log \mathbf{y}_{i,c},$$

where:

- C denotes the number of classes.
- $\mathbf{y}_{i,c}^{\text{true}}$ is the ground-truth one-hot label for node v_i.
- $\mathbf{y}_{i,c}$ corresponds to the predicted probability.

V. Experiment

In this section, we detail the full configuration of our experiments and the corresponding results obtained. These experiments are designed to comprehensively evaluate the performance of our proposed model against existing methods.

A. Dataset

In our experiments, we utilize four well-known datasets: ISCX-VPN2016 [38], ISCX-Tor2016 [39], USTC-TFC2016 [9], and MIRAGE-2024 [40].

The ISCX-VPN2016 dataset is specifically designed for the analysis of Virtual Private Network (VPN) traffic and non-VPN traffic. It contains network traffic from both VPN and non-VPN connections, making it suitable for researching and comparing the characteristics and behaviors of these two types of traffic. The dataset defines six categories, namely Chat, Email, File Transfer, P2P, Streaming and VoIP.

The ISCX-Tor2016 dataset focuses on traffic anonymized through The Onion Router (Tor) network. It contains ISCX-Tor and ISCX-nonTor datasets. The dataset defines eight categories, namely Browsing, Email, Chat, Audio-streaming, Video-streaming, File Transfer, VoIP, and P2P.

The USTC-TFC2016 dataset is specifically designed for the analysis of normal and abnormal network traffic. It contains two primary categories of traffic: normal traffic and abnormal traffic, making it suitable for researching and distinguishing the characteristics between legitimate and malicious network behaviors. The normal traffic in this dataset covers ten sub-categories (e.g., MySQL, Weibo), while the abnormal traffic also includes ten sub-categories (e.g., Htbot, Zeus).

The MIRAGE-2024 dataset is specifically designed for the analysis of mobile application traffic in real-world scenarios. It contains fine-grained bidirectional flows collected from Android devices, covering a diverse set of popular apps such as ClashRoyale, Discord, WhatsApp, Zoom, and many others, making it suitable for researching encrypted traffic classification and mobile app fingerprinting.

B. Experimental Setting

1) Evaluation Metrics: To quantify the performance of the classification model comprehensively, four key metrics are employed: accuracy, precision, recall, and F1 score. Each metric captures distinct aspects of the model's behavior, as defined below:

a) Accuracy: The percentage of correct predictions out of the total samples.

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}$$

b) Precision: The probability that a sample is actually positive among all those predicted to be positive.

$$\text{Precision} = \frac{TP}{TP + FP}$$

c) Recall: The probability that a sample is predicted to be positive among all those that are actually positive.

$$\text{Recall} = \frac{TP}{TP + FN}$$

d) F1 score: The F1 score is used to provide a comprehensive evaluation of Precision and Recall, and it is the harmonic mean of both precision and recall.

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:17 UTC from IEEE Xplore. Restrictions apply.

TABLE I
HYPERPARAMETER SELECTION OF BPF-GNN VIA GRID SEARCH

Hyperparameters	Grid Search Candidate Values	Optimal Selection
Training Epochs	[20, 30, 40, 50, 60]	30
Batch Size	[32, 64, 96, 128]	32
Learning Rate	[0.0001, 0.001, 0.005, 0.01]	0.01
Optimizer	[Adam, SGD]	Adam
Warm Up Ratio	[0.0, 0.1, 0.2]	0.1
Dropout Rate	[0.0, 0.2, 0.3, 0.5]	0.2
LR Minimum	[1e-5, 5e-5, 1e-4]	1e-4

Note: All hyperparameters are determined via grid search and cross-validation.

While accuracy provides a global view, precision and recall focus on type I and type II errors, respectively. The F1 score integrates both to mitigate biases caused by class imbalance.

2) Experimental Environment: We employ DGL as the graph neural network framework for our model and the baseline models, and each experiment was independently run five times with the average value taken as the experimental evaluation result. All of the models presented in this paper are solved by DGL running on a PC with Intel® Core™ i9-10850K CPU 3.60 GHz and NVIDIA RTX 3080 GPU.

3) Model Parameter Selection: In this section, we elaborate on the hyperparameter configuration for our model. To ensure the model achieves optimal performance, we adopt a grid search strategy to exhaustively evaluate the performance of different hyperparameter combinations within reasonable ranges.

Specifically, we predefine candidate value ranges for core hyperparameters (e.g., training epochs from 20 to 60, batch size from 32 to 128, learning rate from 0.0001 to 0.01). For each combination, we conduct independent training and validation on the dataset, and select the optimal configuration by comprehensively balancing classification accuracy, training efficiency, and model generalization.

For the optimization strategy, we compare the performance of Adam and SGD optimizers via grid search and ultimately select Adam for its superior stability in gradient descent. We also introduce a warm-up stage (scaled to 0.1 of total training steps) to mitigate the impact of large initial learning rates, paired with a cosine annealing scheduler to gradually reduce the learning rate to a minimum of 1e-4. Additionally, dropout layers (set to 0.2) are incorporated to prevent overfitting. The final hyperparameter settings (shown in Table I) are confirmed via repeated grid search validation and cross-validation on the dataset.

C. Model Sensitivity Analysis

We conduct sensitivity analysis on four key hyperparameters that directly affect model performance: the maximum number of packets per flow (K), the maximum length of packet header bytes (m), the maximum length of payload bytes (n), and the k value in Top-k for flow-level graph construction. We select accuracy and F1-score as evaluation metrics to quantitatively assess the impact of parameter variations on classification performance.

1) Impact of the Maximum Number of Packets Per Flow (K): The number of packets per flow directly determines both the count of byte-level graphs and the number of nodes in the packet-level graphs. As shown in Fig. 6a, as K increases, the model's accuracy and F1-score first show a upward trend and then enter a state of fluctuation: when $K = 15$, accuracy and F1-score hit their highest values; once K exceeds 15, the metrics only fluctuate slightly without any notable improvement. This indicates that 15 packets already contain enough information to enable effective learning, while extra packets merely add redundant data.

2) Impact of the Maximum Length of Packet Header Bytes (m): The packet header contains critical control information, and its effective extraction is essential for distinguishing encrypted traffic patterns. The sensitivity analysis results are presented in Fig. 6b. As shown in the figure: When the header byte length m is set to 30, the F1-score remains at a relatively low level, as the truncated header bytes lose key information. As m increases to 50, both the F1-score and accuracy reach their optimal levels. This length fully covers the core control fields while avoiding redundant noise from irrelevant header fields. When m exceeds 50, the F1-score does not improve further and only fluctuates slightly. This result indicates that excessive header bytes do not enhance feature representation; instead, they may introduce redundant information and cause minor performance fluctuations.

3) Impact of the Maximum Length of Payload Bytes (n): The payload carries the core data of encrypted traffic, and its byte length directly impacts the richness of semantic information for packet-level representation learning. As shown in Fig. 6c, with the increase of n, the model's performance first rises to a peak and then gradually declines: when $n = 200$, both accuracy and F1-score reach their maximum values. This length balances the need for effective semantic information acquisition and noise control, enabling the model to effectively capture relationships between bytes in the encrypted payload. When n exceeds 200, accuracy and F1-score gradually decrease. This is because encrypted payloads contain a large number of redundant encrypted bits; excessive byte truncation introduces irrelevant noise, interfering with the PMI-based byte correlation calculation and subsequent graph representation learning.

4) Impact of k Value in Top-k for Flow-Level Graph Construction: The k value determines the number of similar flow neighbors connected to each flow node in the flow-level graph, directly affecting the richness of flow-level feature fusion. As shown in Fig. 6d, with the increase of k, the model's F1-score first increases and then decreases significantly. When $k = 1$, the F1-score is relatively low because of the sparse edge structure of the flow-level graph. Each flow can only aggregate limited similar neighbor information and cannot fully leverage inter-flow similarity for feature enhancement. When $k = 3$, the F1-score reaches its peak. This configuration balances the density of the flow-level graph and the relevance of neighbor information, enabling each flow to aggregate features from the three most similar flows while avoiding the dilution of discriminative features caused by excessive irrelevant neighbors. When k exceeds 3, the F1-score decreases significantly. This

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:17 UTC from IEEE Xplore. Restrictions apply.

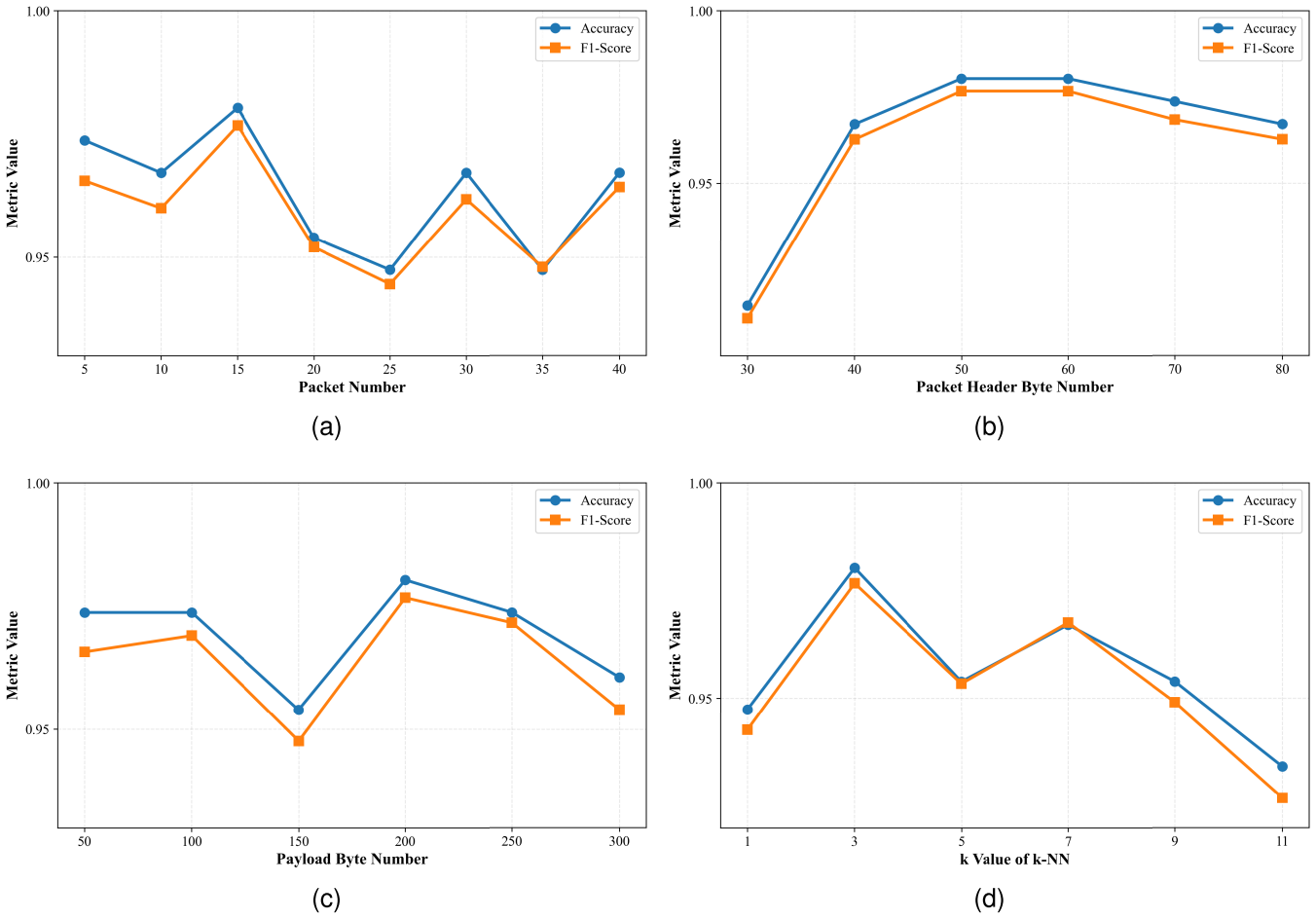


Fig. 6. Sensitivity analysis of key hyperparameters on model performance based on the ISCX-VPN dataset: (a) Maximum number of packets per flow K; (b) Maximum header byte length per packet m; (c) Maximum payload byte length per packet n; (d) k value of Top-k in flow-level graph construction.

is because an overly large k leads to over-connection in the flow-level graph, introducing dissimilar flow information that contaminates the original flow representation.

D. Comparison Experiments

1) Comparison Models: To evaluate the proposed model's efficacy, five baselines are selected for benchmarking. Each method is meticulously optimized for peak performance on the evaluation dataset to ensure fairness. The baseline approaches are summarized as follows:

- **CNN** [9]: Translates the first 784 bytes of each bidirectional flow into grayscale images and use a CNN architecture similar to LeNet-5 [41] for feature learning.
- **KNN-GNN** [12]: Constructs graph representations via K-Nearest Neighbors and integrates graph convolution with an autoencoder for feature extraction.
- **GraphDApp** [13]: Constructs graph structure using packet length sequence and interaction information between packets, with a three-layer GraphSAGE stack used as the neural network model.
- **ACID** [42]: Leverages lightweight neural model-learned low-dimensional embeddings, combined with multiple kernel networks, to adaptively cluster and classify malicious traffic.
- **EC-GCN** [14]: Designs a multi-subgraph classification framework that only uses metadata information, combining CNN and GNN to extract multi-scale spatiotemporal features for encrypted traffic classification.

2) Experimental Results: The comparative experimental results on the ISCX-VPN2016, ISCX-Tor2016, USTC-TFC2016, and MIRAGE-2024 datasets are shown in Tables II and III. Based on the results, we can draw the following conclusions:

1) Our model BPF-GNN performs the best compared to multiple baselines on the ISCX-VPN2016, ISCX-Tor2016, USTC-TFC2016, and MIRAGE-2024 datasets, which confirms the effectiveness of our approach;
2) BPF-GNN outperforms the CNN model across all datasets, and this performance gap essentially stems from the difference in their adaptability to the inherent structure of traffic data: the CNN model converts traffic data into grid structures in Euclidean space for processing, while network traffic inherently exhibits non-Euclidean interaction characteristics. This limits the CNN's ability to extract effective features. In contrast, by constructing a three-level graph structure, BPF-GNN naturally aligns with the non-Euclidean nature of traffic data, enabling accurate modeling of key information

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:17 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISON OF CLASSIFICATION PERFORMANCE ON THE ISCX-VPN2016, ISCX-TOR2016 AND USTC-TFC2016 DATASETS

Task	ISCX-VPN		ISCX-nonVPN		ISCX-Tor		ISCX-nonTor		USTC-TFC2016	
Method	Accuracy	F1-Score	Accuracy	F1-Score	Accuracy	F1-Score	Accuracy	F1-Score	Accuracy	F1-Score
CNN	0.8553	0.8608	0.7772	0.7871	0.6831	0.5314	0.9087	0.7811	0.9840	0.9825
KNN-GNN	0.7829	0.7818	0.7797	0.7698	0.4648	0.2023	0.8612	0.7093	0.8489	0.8182
GraphDApp*	0.9079	0.9073	0.7468	0.7473	0.7042	0.5429	0.9062	0.7567	0.8996	0.8927
ACID	0.7895	0.7796	0.7165	0.7095	0.9225	0.9042	0.8740	0.6959	0.9403	0.9377
EC-GCN	0.8289	0.8252	0.7418	0.7398	0.6690	0.5321	0.8997	0.7397	0.8123	0.8069
BPF-GNN	**0.9803**	**0.9767**	**0.9342**	**0.9394**	**0.9859**	**0.9791**	**0.9614**	**0.8988**	**0.9978**	**0.9967**

**Note:* GraphDApp is originally designed for decentralized applications, leading to limited classification performance on the datasets in this paper. For fair comparison, we replace its original neural network with stacked GraphSAGE while retaining its core graph construction method.

such as byte semantic correlations, packet interaction patterns, and inter-flow similarity;

3) Additionally, it is worth noting that our model is superior to GraphDApp, for which feature extraction only stays at the packet level and does not utilize the raw byte information of the traffic as node features, instead choosing to use packet length information. This precisely confirms the superiority of hierarchical feature extraction;
4) BPF-GNN demonstrates superior generalization capability in cross-scenario tasks across multiple datasets. Experimental results indicate that BPF-GNN maintains consistent and leading classification performance across four datasets with distinctly different characteristics. This advantage originates from its multi-granularity feature extraction mechanism, which can effectively capture both commonalities and scenario-specific features under various encrypted traffic scenarios.

E. Ablation Experiments

In this paper, we conducted a series of ablation experiments aimed at evaluating the impact of each component of the model. Ablation experiments are an important research method that determines the contribution of each part to the final results by systematically removing or replacing various components of the model. In this experiment, we particularly focused on the impact of byte-level graph, packet-level graph, flow-level graph, and the choice of activation functions on the model. Below is an explanation of each experiment:

- **w/o Byte-level graph:** Instead of constructing byte-level graphs, the raw bytes of the traffic data are directly used as node vectors in the packet-level graph.
- **w/o Packet-level graph:** Instead of constructing packet-level graphs, the packet-level features output from the byte-level graph feature extraction are simply concatenated to serve as the node vectors of the flow-level graph.
- **w/o Flow-level graph:** Instead of constructing flow-level graphs, the output from the packet-level graph feature extraction is directly input into a fully connected classifier.
- **w/o PReLU & BatchNorm:** Instead of using the PReLU activation function and normalization, the ReLU activation function is used.

TABLE III
COMPARISON OF CLASSIFICATION PERFORMANCE ON THE MIRAGE-2024 DATASET

Model	Accuracy	Precision	Recall	F1-Score
CNN	0.7747	0.7866	0.7684	0.7729
KNN-GNN	0.6867	0.7332	0.6803	0.6935
GraphDApp*	0.7790	0.8176	0.7751	0.7870
ACID	0.6229	0.6329	0.6064	0.6119
EC-GCN	0.6349	0.6807	0.6308	0.6372
BPF-GNN	**0.8637**	**0.8816**	**0.8462**	**0.8539**

The results of the ablation experiments on the ISCX-VPN2016 dataset are shown in Table IV. The experimental results indicate that the four components aforementioned of BPF-GNN contribute positively to improving the model performance. Among them, the negative impact of not using the byte-level graph on the model performance is the most significant, with the F1 score dropping by approximately 12% on the ISCX-VPN dataset. This illustrates the necessity of starting feature learning from the byte-level. Additionally, we can also observe that not using the packet-level or flow-level graphs also has a certain negative impact on the model performance. Specifically, the flow-level graph enriches the feature representation of each flow by incorporating information from similar flows, thus enhancing the feature representation of the flow, which is confirmed in this section.

F. Model Complexity Analysis

To fully evaluate the balance between performance and computational overhead, we report the floating-point operations (FLOPs) and parameter counts of the proposed BPF-GNN model and baseline methods in Table V.

Combining the comparisons between Table V and performance results, it can be concluded that BPF-GNN achieves competitive performance on the target dataset while maintaining moderate model complexity. Compared with lightweight models such as CNN, GraphDApp and ACID, BPF-GNN has relatively higher floating-point operations. This computational overhead stems from two core design choices:

- **Hierarchical raw byte input:** BPF-GNN takes raw byte sequences as input while leveraging the hierarchical structure of network traffic. This design enhances

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:17 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
ABLATION EXPERIMENTS ON THE ISCX-VPN2016 DATASET

Task	ISCX-VPN				ISCX-nonVPN			
Method	Accuracy	Precision	Recall	F1-Score	Accuracy	Precision	Recall	F1-Score
w/o Byte-level graph	0.8553	0.8728	0.8366	0.8488	0.7772	0.7959	0.7985	0.7955
w/o Packet-level graph	0.9605	0.9700	0.9409	0.9507	0.8987	0.9084	0.8937	0.8995
w/o Flow-level graph	0.9737	0.9769	0.9665	0.9702	0.9013	0.9135	0.9073	0.9083
w/o PR & BN	0.9474	0.9430	0.9395	0.9407	0.8228	0.8152	0.8235	0.8146
Default	**0.9803**	**0.9844**	**0.9722**	**0.9767**	**0.9342**	**0.9392**	**0.9411**	**0.9394**

TABLE V
MODEL FLOPS AND PARAMETERS

Model	FLOPS(M)	Parameters(M)
CNN	1.39e+01	3.27e+00
KNN-GNN	8.66e+00	8.67e+00
GraphDApp*	1.94e+00	1.83e-01
ACID	1.88e-01	1.89e-01
EC-GCN	7.07e+02	6.55e+02
BPF-GNN	**4.58e+02**	**1.56e+00**

feature representation capability but also increases computational overhead. However, by adjusting the number of bytes used per flow, the floating-point operations can be reduced with acceptable accuracy loss.

- **Separate processing of packet header and payload:** BPF-GNN processes packet headers and payloads via independent neural network branches. This design strengthens feature learning for encrypted traffic but doubles the computational load. This overhead, though, can be mitigated through parallel processing.

VI. CONCLUSION

In this paper, we propose BPF-GNN, a hierarchical graph neural network model designed for encrypted traffic classification through multi-granularity feature extraction across byte-, packet-, and flow-level graphs. This allows us to capture the discriminative information contained at the byte, packet, and flow levels, thereby enriching the representation of the flow-level features used for classification and improving the accuracy of encrypted traffic classification tasks. Experimental results on the public datasets demonstrate that our method achieves significant classification performance. This remarkable performance can be attributed to the effectiveness of the multi-granularity feature extraction approach. In future work, we will continue to explore the impact of replacing different graph construction methods at each granularity level on the final classification results.

REFERENCES

[1] S. Rezaei and X. Liu, "Deep learning for encrypted traffic classification: An overview," *IEEE Commun. Mag.*, vol. 57, no. 5, pp. 76–81, May 2019.

[2] T. Nelms, R. Perdisci, and M. Ahamad, "ExecScent: Mining for new C&C domains in live networks with adaptive control protocol templates," in *Proc. 22nd USENIX Secur. Symp. (USENIX Secur.)*, 2013, pp. 589–604.

[3] W. Melo, P. Lopes, R. Antonello, S. Fernandes, and D. Sadok, "On the performance of DPI signature matching with dynamic priority," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Jun. 2014, pp. 1–6.

[4] J. Su, S. Chen, B. Han, C. Xu, and X. Wang, "A 60Gbps DPI prototype based on memory-centric FPGA," in *Proc. ACM SIGCOMM Conf.*, Aug. 2016, pp. 627–628.

[5] A. B. Mohd and S. bin Mohd Nor, "Towards a flow-based internet traffic classification for bandwidth optimization," *Int. J. Comput. Sci. Secur.*, vol. 3, no. 2, pp. 146–153, 2009.

[6] B. Anderson and D. McGrew, "Machine learning for encrypted malware traffic classification: Accounting for noisy labels and non-stationarity," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2017, pp. 1723–1732.

[7] B. Xu, S. Chen, H. Zhang, and T. Wu, "Incremental k-NN SVM method in intrusion detection," in *Proc. 8th IEEE Int. Conf. Softw. Eng. Service Sci. (ICSESS)*, Nov. 2017, pp. 712–717.

[8] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *Proc. IEEE Int. Conf. Intell. Secur. Informat. (ISI)*, Jul. 2017, pp. 43–48.

[9] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Jan. 2017, pp. 712–717.

[10] T.-L. Huoh, Y. Luo, P. Li, and T. Zhang, "Flow-based encrypted network traffic classification with graph neural networks," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 2, pp. 1224–1237, Jun. 2023.

[11] J. Zheng and D. Li, "GCN-TC: Combining trace graph with statistical features for network traffic classification," in *Proc. IEEE Int. Conf. Commun. (ICC)*, May 2019, pp. 1–6.

[12] B. Sun, W. Yang, M. Yan, D. Wu, Y. Zhu, and Z. Bai, "An encrypted traffic classification method combining graph convolutional network and autoencoder," in *Proc. IEEE 39th Int. Perform. Comput. Commun. Conf. (IPCCC)*, Nov. 2020, pp. 1–8.

[13] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.

[14] Z. Diao et al., "EC-GCN: A encrypted traffic classification framework based on multi-scale graph convolution networks," *Comput. Netw.*, vol. 224, Apr. 2023, Art. no. 109614.

[15] H. Zhang et al., "TFE-GNN: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification," in *Proc. ACM Web Conf.*, Apr. 2023, pp. 2066–2075.

[16] A. Finamore, M. Mellia, M. Meo, and D. Rossi, "KISS: Stochastic packet inspection classifier for UDP traffic," *IEEE/ACM Trans. Netw.*, vol. 18, no. 5, pp. 1505–1515, Oct. 2010.

[17] F. Zaki, F. Afifi, S. A. Razak, A. Gani, and N. B. Anuar, "GRAIN: Granular multi-label encrypted traffic classification using classifier chain," *Comput. Netw.*, vol. 213, Aug. 2022, Art. no. 109084.

[18] H. Mohanty, A. H. Roudsari, and A. H. Lashkari, "Robust stacking ensemble model for darknet traffic classification under adversarial settings," *Comput. Secur.*, vol. 120, Sep. 2022, Art. no. 102830.

[19] X. Yun, Y. Wang, Y. Zhang, and Y. Zhou, "A semantics-aware approach to the automated network protocol identification," *IEEE/ACM Trans. Netw.*, vol. 24, no. 1, pp. 583–595, Feb. 2016.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:17 UTC from IEEE Xplore. Restrictions apply.

[20] Y. Wang, X. Yun, and Y. Zhang, "Rethinking robust and accurate application protocol identification: A nonparametric approach," in *Proc. IEEE 23rd Int. Conf. Netw. Protocols (ICNP)*, Nov. 2015, pp. 134–144.
[21] X. Xiao, R. Li, H.-T. Zheng, R. Ye, A. KumarSangaiah, and S. Xia, "Novel dynamic multiple classification system for network traffic," *Inf. Sci.*, vol. 479, pp. 526–541, Apr. 2019.
[22] P. Tang, Y. Dong, and S. Mao, "Online traffic classification using granules," in *Proc. IEEE INFOCOM Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, Jul. 2020, pp. 1135–1140.
[23] A. L'Heureux, K. Grolinger, H. F. Elyamany, and M. A. M. Capretz, "Machine learning with big data: Challenges and approaches," *IEEE Access*, vol. 5, pp. 7776–7797, 2017.
[24] Z. Bu, B. Zhou, P. Cheng, K. Zhang, and Z.-H. Ling, "Encrypted network traffic classification using deep and parallel network-in-network models," *IEEE Access*, vol. 8, pp. 132950–132959, 2020.
[25] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, Apr. 2019, pp. 1171–1179.
[26] X. Ren, H. Gu, and W. Wei, "Tree-RNN: Tree structural recurrent neural network for network traffic classification," *Expert Syst. Appl.*, vol. 167, Apr. 2021, Art. no. 114363.
[27] J. Yang, X. Jiang, Y. Lei, W. Liang, Z. Ma, and S. Li, "MTSecurity: Privacy-preserving malicious traffic classification using graph neural network and transformer," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 3, pp. 3583–3597, Jun. 2024.
[28] Y. Shi, G. Li, and J. Wu, "BPF-DAG: Byte-packet-flow fatures fusion via dynamic attributed graph for reliable encrypted traffic classification," *IEEE Trans. Inf. Forensics Security*, vol. 21, pp. 197–211, 2025.
[29] X. Xiao, W. Xiao, R. Li, X. Luo, H.-T. Zheng, and S. Xia, "EBSNN: Extended byte segment neural network for network traffic classification," *IEEE Trans. Depend. Sec. Comput.*, vol. 19, no. 5, pp. 3521–3538, Aug. 2021.
[30] X. Xiao et al., "RBLJAN: Robust byte-label joint attention network for network traffic classification," *IEEE Trans. Depend. Secure Comput.*, vol. 22, no. 3, pp. 2161–2178, May 2025.
[31] L. Yao, C. Mao, and Y. Luo, "Graph convolutional networks for text classification," in *Proc. AAAI Conf. Artif. Intell.*, 2019, vol. 33, no. 1, pp. 7370–7377.
[32] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1024–1034.
[33] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Dec. 2015, pp. 1026–1034.
[34] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 448–456.
[35] K. Xu, C. Li, Y. Tian, T. Sonobe, K. Kawarabayashi, and S. Jegelka, "Representation learning on graphs with jumping knowledge networks," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5453–5462.
[36] J. Arevalo, T. Solorio, M. Montes-y-Gómez, and F. A. González, "Gated multimodal units for information fusion," 2017, *arXiv:1702.01992*.
[37] K. Al-Naami et al., "Adaptive encrypted traffic fingerprinting with bi-directional dependence," in *Proc. 32nd Annu. Conf. Comput. Secur. Appl.*, Dec. 2016, pp. 177–188.
[38] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in *Proc. 2nd Int. Conf. Inf. Syst. Secur. Privacy*, 2016, pp. 407–414.
[39] A. H. Lashkari, G. D. Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor traffic using time based features," in *Proc. 3rd Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, Porto, Portugal, Feb. 2017, pp. 253–262.
[40] I. Guarino, D. Ciuonzo, A. Montieri, and A. Pescapè, "Mirage-App×Act-2024: A novel dataset for mobile app and activity traffic analysis," in *Proc. 20th Int. Conf. Wireless Mobile Comput., Netw. Commun. (WiMob)*, Oct. 2024, pp. 663–666.
[41] Y. LeCun et al., "Learning algorithms for classification: A comparison on handwritten digit recognition," *Neural Netw., Stat. Mech. Perspective*, vol. 56, no. 3, pp. 261–276, 1995.
[42] A. F. Diallo and P. Patras, "Adaptive clustering-based malicious traffic classification at the network edge," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2021, pp. 1–10.



Guolong Li is currently pursuing the Ph.D. degree with Information Engineering University. His research interests include encrypted traffic analysis.



Yuan Gao received the Ph.D. degree from Shandong University. She is currently a Lecturer with Information Engineering University. Her research interests include cryptography and encrypted traffic detection.



Jiongjiong Ren received the Ph.D. degree from Information Engineering University. He is currently an Associate Professor with Information Engineering University. His research interests include cryptography and its applications.



Shaozhen Chen received the Ph.D. degree from Shandong University. She is currently a Professor with Information Engineering University. Her research interests include cryptography and information security.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:40:17 UTC from IEEE Xplore. Restrictions apply.